



MOTIRON
Technologies

PROJETO DA RESIDÊNCIA
Plataforma de Gestão

1. CONTEXTO DO PROJETO

Durante a mentoria, cada squad deverá desenvolver o frontend de uma plataforma web de gestão corporativa.

O cenário é o de uma empresa fictícia com múltiplas unidades, que precisa acompanhar documentos, tarefas e indicadores de cada uma delas em um único painel centralizado.

O objetivo é construir uma aplicação funcional que simule uma plataforma real de gestão, aplicando práticas de arquitetura, organização de código e experiência de uso adotadas no mercado de desenvolvimento de software.

2. STACK OBRIGATÓRIA

O projeto deve ser desenvolvido utilizando **obrigatoriamente**:

- Next.js
- TypeScript

Bibliotecas recomendadas: Tailwind CSS, React Hook Form, Zod, TanStack Query e TanStack Table.

3. ESTRUTURA DO SISTEMA

- **Tela de Login**: acesso com e-mail e senha, validação dos campos e feedback visual de sucesso ou erro.
- **Dashboard**: exibe indicadores gerais — total de unidades, documentos e tarefas pendentes — e uma visão rápida da situação da empresa.
- **Lista de Unidades**: página com todas as unidades cadastradas, com pesquisa, filtros e paginação.
- **Detalhamento da Unidade**: mostra informações, documentos e tarefas vinculados à unidade selecionada.
- **Lista de Documentos**: exibe os documentos cadastrados e permite acompanhar seu status de validade.
- **Lista de Tarefas**: acompanhamento das tarefas vinculadas às unidades, com status visual e prazos.

4. REGRAS DE NEGÓCIO

- Autenticação obrigatória para acesso ao sistema; usuários não autenticados devem ser redirecionados para a tela de login.
- Cada unidade possui um status (ativa ou inativa); unidades inativas não podem receber novos documentos ou tarefas.
- Um documento possui status calculado a partir da data de validade: válido, próximo do vencimento (até 15 dias) ou expirado. Esse status não deve ser um valor livre — deve ser derivado da data pelo frontend.
- Uma tarefa possui status Pendente, Em andamento ou Concluída, e não pode ser marcada como concluída sem confirmação do usuário.
- O total de pendências exibido no Dashboard (documentos + tarefas) deve refletir, em tempo real, qualquer alteração feita nas telas de listagem, sem exigir recarregar a página.
- Documentos e tarefas de uma unidade inativa devem ser exibidos como somente leitura no detalhamento.
- A squad deve criar mocks suficientes para demonstrar os diferentes estados descritos acima (unidades ativas/inativas, documentos em cada status, tarefas em cada status).

5. REQUISITOS TÉCNICOS

- Responsividade: a interface deve funcionar corretamente em desktop, tablet e mobile.
- Feedback visual: estados de loading, mensagens de erro e estados vazios devem ser exibidos em todas as telas que dependem de dados.
- Consistência visual: tipografia, cores e espaçamentos consistentes em toda a aplicação, seguindo um design system mínimo definido pela squad.
- Componentização: componentes reutilizáveis (Button, Input, Modal, Table, Badge, etc.), sem duplicação de lógica de UI entre telas.
- Gerenciamento de estado: separação clara entre estado de servidor (dados mockados) e estado de UI (filtros, modais, formulários abertos).
- Tipagem: todas as entidades (Unidade, Documento, Tarefa) devem ter interfaces/types próprios, sem uso de `any`.
- Formulários com validação de schema (ex.: Zod) e mensagens de erro específicas por campo.
- Acessibilidade básica: uso correto de labels, contraste mínimo de cores e navegação por teclado nos formulários.
- Performance: paginação ou virtualização em listas com muitos itens, evitando re-renderizações desnecessárias.

6. ENTREGAS OBRIGATÓRIAS

- Repositório público com código organizado.

- README com descrição do projeto, tecnologias utilizadas, arquitetura e instruções para execução.
- Aplicação funcional executável localmente ou publicada online.

7. ENTREGAS EXTRAS

Não são obrigatórias, mas demonstram maior maturidade técnica e serão consideradas como diferencial na avaliação.

- **Camada de API própria:** criação de rotas (Next.js Route Handlers ou um mock server como MSW) simulando latência e erros de rede, para preparar a troca futura dos mocks por uma API real.
- **Cache e revalidação:** uso de TanStack Query com invalidação de cache entre telas — ex.: concluir uma tarefa no detalhamento da unidade deve refletir automaticamente na lista de tarefas e no Dashboard.
- **Regra de negócio avançada:** bloqueio de exclusão de uma unidade que possua documentos pendentes ou tarefas em andamento, com mensagem explicando o motivo.
- **Auditoria simples:** histórico local de alterações (quem mudou o quê e quando) exibido no detalhamento da unidade.
- **Dark mode:** alternância de tema persistida entre sessões, sem quebrar o contraste dos status visuais.
- **Testes automatizados:** cobertura de pelo menos as regras de negócio (cálculo de status de documento, bloqueios de unidade inativa) com testes unitários.
- **Deploy:** aplicação publicada (Vercel ou similar) com pipeline de build automatizado.

8. AVALIAÇÃO DO PROJETO

A avaliação será composta por três critérios principais:

- **Desempenho Técnico (60%)**

Avalia a qualidade técnica geral da aplicação desenvolvida.

Critérios considerados: implementação correta das funcionalidades solicitadas, organização e arquitetura do projeto, clareza e qualidade do código, reutilização de componentes, consistência da interface, experiência do usuário, responsividade da aplicação e tratamento adequado de estados de erro e carregamento. Também será observado se a aplicação está funcional, estável e próxima de um padrão de qualidade esperado em projetos profissionais de frontend.

- **Processos e Organização** (10%)

Avalia a forma como o projeto foi estruturado e organizado durante o desenvolvimento.

Critérios considerados: organização do repositório, estrutura de pastas do projeto, clareza da documentação, qualidade do README, histórico de commits no Git e clareza das mensagens de commit. A ideia é avaliar se a squad demonstrou boas práticas de organização de projetos e colaboração em equipe, mantendo o código versionado de forma clara e estruturada.

- **Apresentação Final** (30%)

Avalia a capacidade da squad de apresentar e explicar o projeto desenvolvido.

Critérios considerados: clareza da apresentação, demonstração prática do sistema funcionando, organização da explicação do projeto, capacidade de descrever as decisões técnicas tomadas durante o desenvolvimento e domínio geral da solução construída. Também será observado se a equipe consegue demonstrar segurança ao explicar o funcionamento do sistema e responder perguntas relacionadas à implementação.

Níveis de avaliação:

- **Insuficiente** – Nota 4

- **Básico** – Nota 6

- **Bom** – Nota 8

- **Excelente** – Nota 10

9. OBJETIVO DA MENTORIA

Ao final da mentoria, espera-se que os alunos tenham desenvolvido um frontend completo e funcional, com experiência prática em arquitetura moderna de aplicações, regras de negócio, trabalho em squad e capacidade de apresentar e defender suas decisões técnicas.

Mais do que implementar funcionalidades, o objetivo é aprender a construir e entregar produto de qualidade como se faz no mercado real.